

COMP3111 Advanced Web Development
COMP3122 Advanced Web Development
COMP312 Internet Programming II

Teaching Team

2

- Dr. Chak Fong Chong (Max)
- t1968@mpu.edu.mo

Learning Outcomes

3

After completing the learning module, students will be able to:

1. Identify the architecture and operation of the Web
2. Differentiate the major architectural components of modern Web applications and devise how they work together
3. Design Web APIs using established styles
4. Develop Web application clients that have interactive user interfaces
5. Construct scalable, database-backed Web application servers



Key Topics

4

- HTTP operations
- JavaScript programming language
- JavaScript in Browsers (Client-side Programming)
 - ▣ Vue.js
- Node.js (Server-Side JavaScript Programming)
- Web APIs and App Server Programming
- Advanced Topics in HTTP



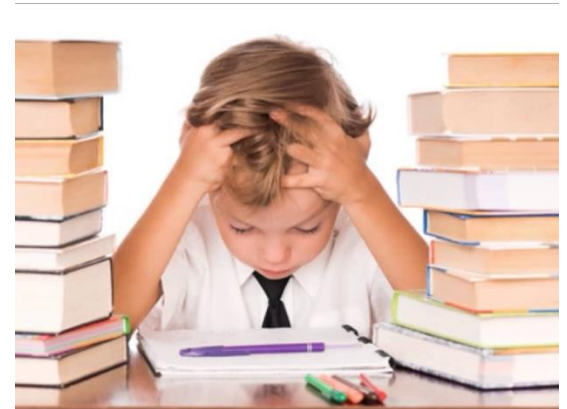
**Attendance will
be taken in
every session**

Assessments

5

- Assignments (40%)
 - ▣ Written Assignment (20%; Due Lesson 7*)
 - ▣ Individual Project (20%; Due Lesson 13*)
- Test (20%; Lesson 8*)
- Final Exam (40%)

** Dates are tentative and may be adjusted based on the teaching progress*



Advices on Assignment and Project

6

- Ask questions early. Don't wait until the last minute!
- Participate in group work
- Practice! Practice! Practice!
- Do not rely on ChatGPT or other AI
- **No plagiarism and academic misconduct!**
 - To deal with plagiarism, a zero mark will be given if you copy someone else's work or you let someone copy your work
- Read the use of **Generative AI policy** carefully!



HTTP Operation and Message Format

Outline

8

- A. HTTP basics
 - ▣ URL, MIME-type, and HTTP message
- B. HTTP methods: GET vs. POST
- C. Redirection

Part A. HTTP basics

9

- Web browsers, web servers
 - ▣ Web proxies (to be discussed in chap 6)
- Web resources
 - ▣ URL to locate a resource
 - ▣ MIME-type to specify data type
- HTTP
 - ▣ Message format: common headers and status code
- HTTP request and response

Web browsers and servers

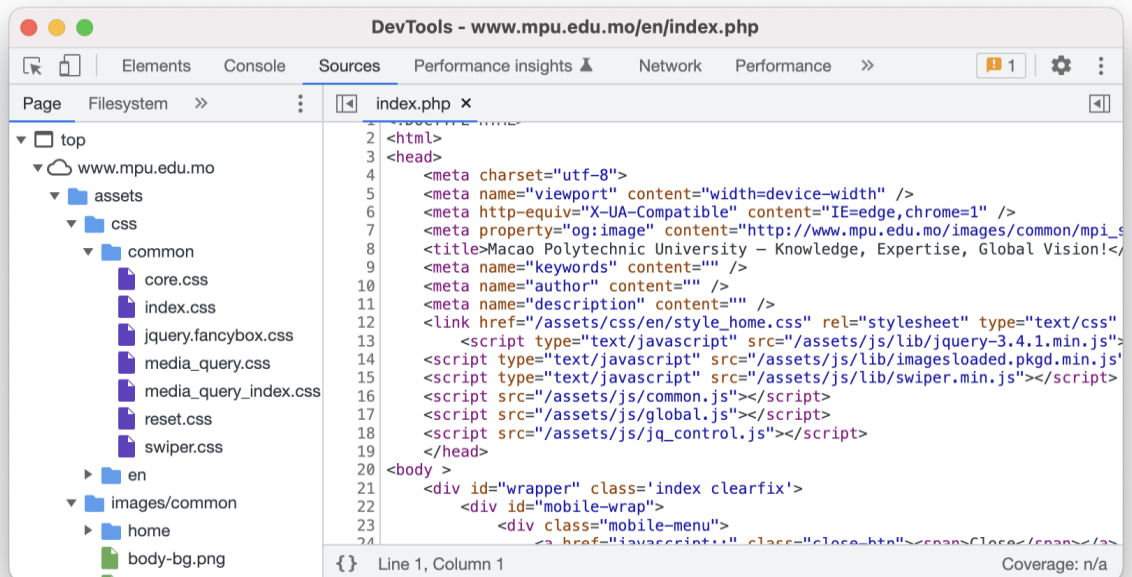
10

- A **web browser** is a program to retrieve and display resources on the Web
- Popular web browsers: Microsoft Edge, Mozilla Firefox, Google Chrome, Apple Safari
- A **web server** receives requests from a web browser and returns the requested resource.
 - ▣ Retrieve the resource from file system on server, or
 - ▣ Run a program to generate the resource
- Popular web servers: Apache HTTP server, Microsoft IIS, nginx

Web resources in a web page

11

- A web page actually consists of several kinds of web resources
 - ▣ One or more HTML document
 - ▣ Some CSS style sheets
 - ▣ Some JavaScript programs
 - ▣ Images
 - ▣ Objects
(e.g. flash movies)
 - ▣ Font
 - ▣ ... etc.



Web resources

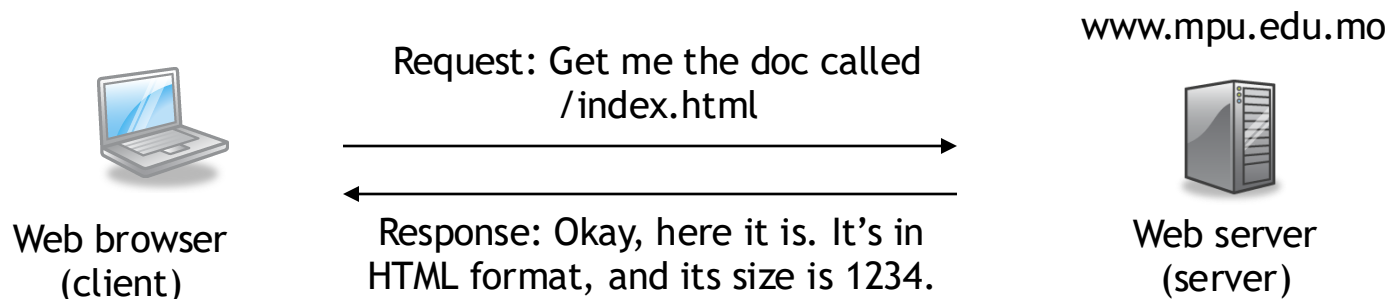
12

- A **resource** is anything that is important enough to be referenced as a thing in itself.
- A resource has at least one **URL** as its address. A browser uses the URL to download the resource from a server.
- The web server uses **MIME type** to specify the data type of a representation of a resource.
- Examples of resources:
 - The front page of MPU web site
 - The logo of MPU
 - A directory of resources pertaining to “web design” found by Google.
 - A JSON record returned by Facebook graph API: <https://graph.facebook.com/cocacola>

Web clients and Web servers

13

- Web browsers and web servers work in a client/server architecture
 - ▣ A browser sends a **request** for a certain resource to the server.
 - ▣ The server returns a **response** which contains the requested resource.
 - ▣ Each transaction consists of *one* request and *one* response
- **Each request retrieves one resource only**
 - ▣ Note: loading a typical web page requires many HTTP requests



A browser retrieves `http://www.mpu.edu.mo/index.html`

Address of web resource

14

- Each web resource has a **URL** (Uniform Resource Locator) as an address. It includes
 - ▣ address of a web server – where to find the resource
 - ▣ protocol – how to communicate with the server
 - ▣ additional info for the server to find the resource – which resource in the server
- Given an URL, a browser has enough info to construct a request to retrieve the resource

Syntax of URL

15

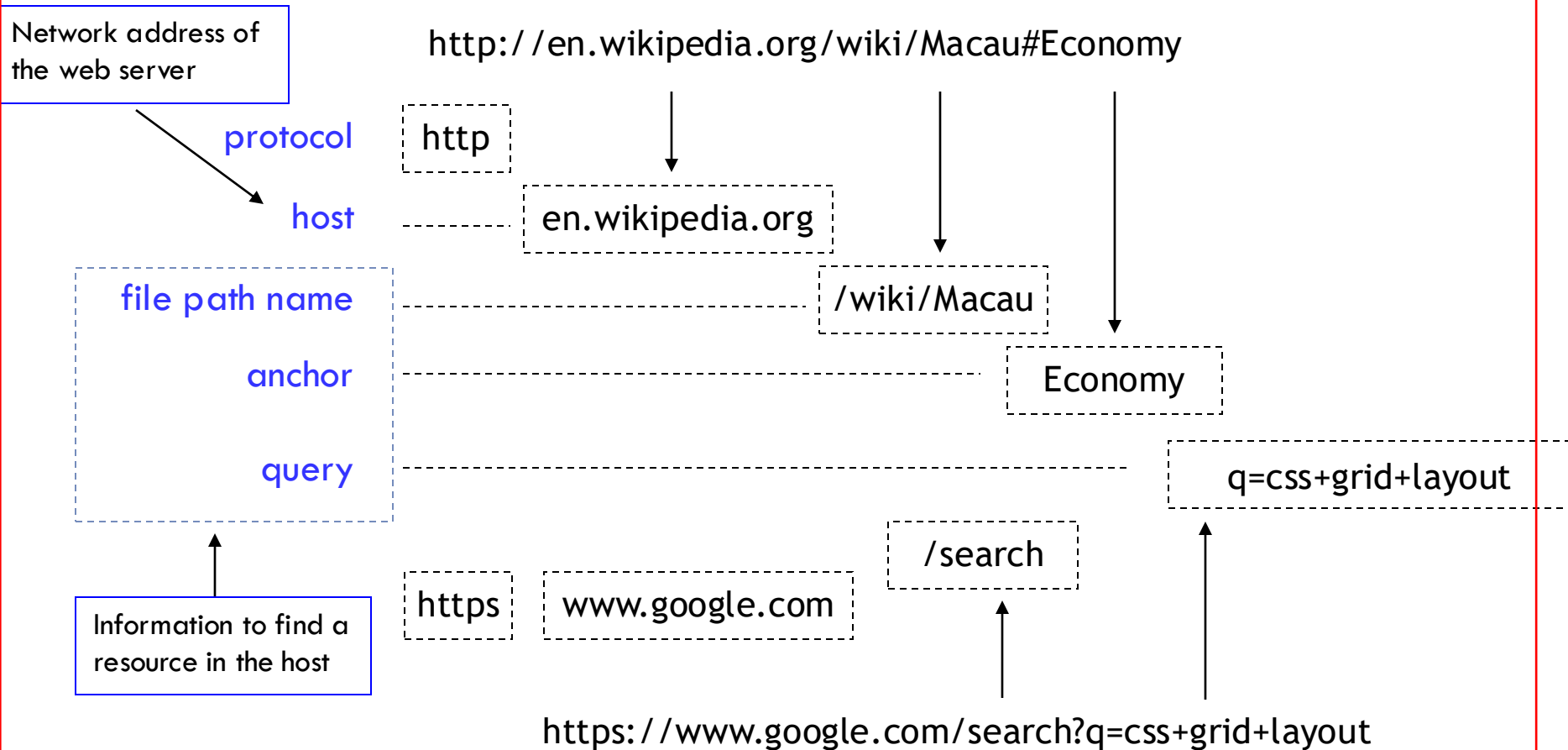
protocol://host/filepathname?query#anchor

- ▣ **Protocol** is either 'http' or 'https'
- ▣ **Host** is usually a domain name of a web server
- ▣ **File path name** identifies a resource in the server
- ▣ **Query** is form data submitted to a server-side script.
The format is *name1=value1&name2=value2*
- ▣ **Anchor** refers to an element with the id in an html or xml file.
- ▣ Notice that separator characters in **blue** must be encoded if they are used as ordinary text in URL.

Inside an URL ...

EXAM / ASSIGNMENT

16



Encode characters in URL

17

- In general, characters other than letters and numbers should be encoded in URL.
 - ▣ Space may be written as '+' or '%20'
 - ▣ Other printable character in ASCII should be written in hexadecimal. E.g 1/2 is encoded as '1%2F2'
 - ▣ Non-ascii characters, e.g. chinese, should be written as utf-8 in hexadecimal. E.g. 中文 is encoded as '%E4%B8%AD%E6%96%87'
 - ▣ Experiment: check how a browser encodes data in form submission.

Data type of web resource

18

- There are various types of web resources
 - ▣ E.g. text, image, audio, video, data
- There are different formats to encode a certain type
 - ▣ E.g. an image can be in GIF, JPEG or PNG
 - ▣ Some formats are defined by W3C, e.g. HTML, CSS, XML (general data), PNG (bitmap image), SVG (vector graphics)
 - ▣ Others are de-facto standards defined by the industry, e.g. GIF, JPEG, SWF (flash movie), JavaScript
- These formats are identified by a standard called **MIME type**
- Most browsers support these formats, and can display them correctly.

https://en.wikipedia.org/wiki/Media_type

Common MIME types

19

EXAM

category	MIME type	format
Text	text/html	HTML (.html)
	text/css	CSS (.css)
Images	image/gif	GIF (.gif)
	image/jpeg	JPEG (.jpg)
	image/png	PNG (.png) Portable Network Graphics
	image/svg+xml	SVG (.svg) Scalable Vector Graphics
Audio	audio/mp3, audio/m4a	Mp3 audio, MPEG4 audio
Video	video/mp4, video/m4v	MPEG4 video
Multipurpose	application/xml	General XML
	application/json	JSON data
	application/javascript	JavaScript source (.js)
	application/octet-stream	Arbitrary binary data
	application/x-www-form-urlencoded	HTML form submission

Character encoding

20

- In addition to the format of a text file, web server should also specify its encoding.
 - Character encoding converts a text character into a sequence of bytes
 - Common encoding: ISO-8859 (Western) (similar to ASCII), Big5 (Traditional Chinese), GB2312 (Simplified Chinese), UTF-8 (Unicode)
 - Example: 中 = (Big5) A4A4 = (Gb) D6D0 = (utf-8) E4B8AD

HTTP Protocol

APPLICATION LAYER

21

- **Hypertext Transfer Protocol (HTTP)** is the protocol used to deliver and perform operations on web resources on the Web.
 - ▣ It defines the meaning and structure of requests and responses, and how to handle error.
 - ▣ HTTP is also used in Ajax and Web API, e.g. Google Search, Facebook Graph API, Amazon Simple Storage Service
- HTTP is a W3C standard
 - ▣ Current version HTTP/1.1 (RFC7230, RFC7231, ... RFC7235)
 - ▣ HTTP/2 published on May 2015 as RFC7540.
 - ▣ HTTP/3 published on June 2022
 - ▣ https = HTTP runs on top of SSL. Encrypt requests and responses for better security
 - ▣ Runs on TCP. Default port is 80 (HTTPS uses port 443)

HTTP operation

22

- Each HTTP transaction consists of one request and one response
 - ▣ A client sends a request for a resource to the server. Then the server returns a response containing a representation of the resource to the client.
- HTTP/1.1 can transfer several web resources in a TCP connection.

Basic retrieval

23

- to retrieve a static file in server
 - A browser sends a request to GET a resource at a URL
 - The server receives the request, maps the URL to a file in its file system, e.g.
`http://example.com/home.html -> c:\inetpub\home.html`
 - The server infers the MIME-type from the file extension, e.g. `*.html -> text/html`, `*.gif -> image/gif`
 - The server constructs and returns the response



Example: Basic retrieval

24

HTTP
request

```
GET /home.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 ... Chrome/52.0...
```



example.com



Simple HTTP retrieval of the page
`http://example.com/home.html`. Notice how the URL
of the resource is specified in the request, and how
the data type is stated in the response.

```
HTTP/1.1 200 OK
Date: Wed, 17 Aug 2016 02:36:26 GMT
Server: Apache
Content-Type: text/html; charset=utf-8

<html>... </html>
```

HTTP response

Basic HTTP request and response

25

- A basic request contains
 - ▣ Method **GET** to **retrieve a resource**
 - ▣ **URL** (or part of URL) – **address of the resource**
 - ▣ Other headers, e.g. name of web browser
- A basic response contains
 - ▣ **Status code**
 - ▣ A representation of the **resource**
 - ▣ its **MIME-type** and **encoding** (for text resource) in Content-type header
 - ▣ Other headers, e.g. name of web server

HTTP message structure

26

- HTTP requests and responses have similar structure
 - ▣ Start line – URL requested, any error
 - ▣ Headers – additional info
 - ▣ Blank line
 - ▣ Body – representation of the resource
- Start line and blank line are required; headers and body are optional.

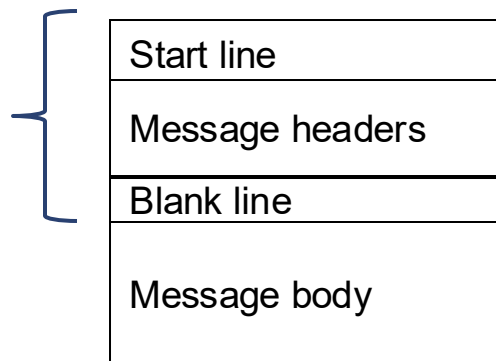
Start line
Message headers
Blank line
Message body

HTTP message structure

27

- Start line and headers must be in US-ASCII
 - ▣ I.e. Chinese characters must be encoded
- Message body can be text in any encoding or binary data
 - ▣ The header Content-Type defines the data type
 - ▣ The header Content-Length defines the size (in bytes)

Each line ends with CR LF (ASCII 13 and 10), and can only contain US-ASCII characters.



The body can be in any encoding or binary data

Request line

28

- The **request line** has three parts:
 - ▣ Method name: GET, POST, HEAD, etc 客户端希望对资源进行的操作
 - ▣ Partial URL of the resource - 是服务器上的资源地址，通常是相对路径
 - ▣ Version of HTTP: HTTP/1.0 or HTTP/1.1
HTTP/1.0 - 早期版本，不支持持久连接

GET /test.html HTTP/1.1

GET /search?hl=en&q=HTTP+headers&btnG=Google+Search HTTP/1.1

POST /accounts/ServiceLoginAuth?service=mail HTTP/1.1

Example: HTTP requests

29

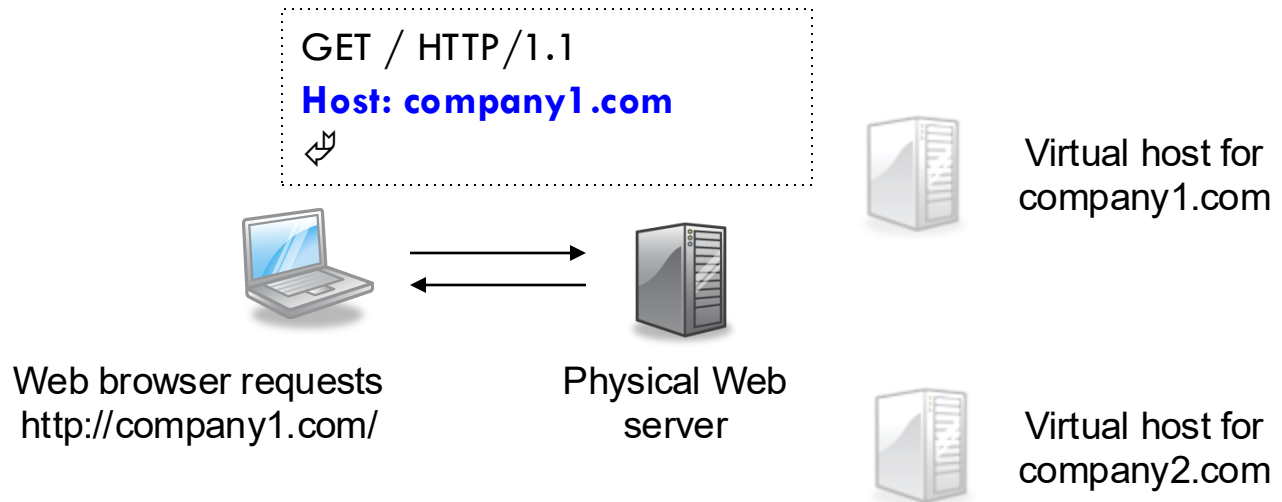
To retrieve the web page `http://www.example.com/test.html`, the web browser constructs an HTTP request and sends it to the web server at `www.example.com`.

```
GET /test.html HTTP/1.1 ↵  
Host: www.example.com ↵  
↵
```

Host:

30

- **Compulsory** header in HTTP/1.1
- Specifies the host of the requested URL
 - Useful for multi-home web servers. Several web sites (with different host names) may be using the **same IP address.**



Status line

31

- The status line has three parts:

- ▣ HTTP version

- ▣ Response status code

- 200 – success

- 404 – file not found

- 500 – server error

- ▣ English reason phrase, which describes the status code

HTTP/1.1 200 OK

HTTP/1.1 404 Not found

- HTTP defines five classes of status code

- ▣ 1xx – informational message

- ▣ 2xx - success of some kind

- ▣ 3xx – redirects the client to another URL

- ▣ 4xx – an error on the client's part

- ▣ 5xx – an error on the server's part

Common codes for success and error

32

Status code	Meaning
200 OK	The server successfully carried out the action that the client requested. For GET request, the response body contains a representation of the requested resource.
204 No content	The server successfully carried out the action, but declined to return any representation. In Ajax, this usually means the browser should not refresh the user interface.
400 Bad request	Generic client-side error. Probably a request format error.
404 Not found	The server cannot find the resource at the requested URL.
500 Internal server error	Generic server-side error. Probably a server-side program run-time error.
503 Service unavailable	The web server is not available, probably because of overloading or maintenance.

<https://tools.ietf.org/html/rfc7231#section-6>

Example: HTTP responses

33

After receiving the request for `http://www.example.com/test.html`, the web server will return a response containing the page, or an error message if it cannot find the page.

```
HTTP/1.1 200 OK ↵  
Content-Type: text/html ↵  
Content-Length: 2300 ↵  
↵  
<html>... content of test.html ...</html>
```

```
HTTP/1.1 404 Not found ↵  
Content-Type: text/html ↵  
Content-Length: 1024 ↵  
↵  
<html>... error message ...</html>
```

Content-Type:

34

- Indicates the data type (MIME) and character encoding of the message body in requests and responses
 - ▣ Omitted if empty body
 - ▣ **Content-Type:** `application/octet-stream` if server cannot decide

HTTP request to login gmail.com

POST /accounts/ServiceLoginAuth?service=mail HTTP/1.1

Host: www.google.com

Content-Type: `application/x-www-form-urlencoded`

Content-Length: 194

...&Email=username&Passwd=password&...

Notice how character encoding is specified.

HTTP response

HTTP/1.1 200 OK

Content-Type: `text/html; charset=UTF-8`

Content-Length: 12800

<html>...</html>

Content-Length:

35

- Indicates the size (in bytes) of the message body in requests and responses
 - ▣ Content-length header is sent before the message body
 - ▣ Difficult to determine message size if the response is generated dynamically by a program
 - ▣ A solution: chunked-transfer encoding (to be discussed)

HTTP request to login gmail.com

POST /accounts/ServiceLoginAuth?service=mail HTTP/1.1

Host: www.google.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 194

...&Email=username&Passwd=password

HTTP response

HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8

Content-Length: 12800

<html>...</html>

Common headers

36

Header	Request	Response
Host:	Domain name of requested URL	
User-agent:	Which web browser?	
Server:		Which web server?
Referer:	From which web page does the browser obtain this URL?	
Content-type:	Type of content in the body	
Content-length:	Length of the body in bytes	

User-Agent: & Server:

37

- Indicate which web browser generates the request and which web server generates the response
 - ▣ May be used for collecting statistics about web browser market share
 - ▣ A server may generate browser specific response

```
GET /test.html HTTP/1.1
Host: csproject.mpu.edu.mo
User-Agent: Mozilla 5.0 (Windows)
...
```

request

```
HTTP/1.1 200 Ok
Server: Apache
...
```

response

Browser and server names in actual use are much longer.

Referer:

38

- From which page the browser obtains the URL of the requested resource
 - E.g. if the user clicks a link to news.html from the page index.html in www.example.com, the browser may send the following request.

```
GET /news.html HTTP/1.1
Host: www.example.com
Referer: http://www.example.com/index.html
...
```

Experiment

39

- Use the network panel in Chrome's developer tools to inspect HTTP requests and responses
- Note that loading a typical web page involves numerous HTTP transactions
- Check the status code in case the URL cannot be found
- Check the headers

Part B. HTTP methods and form submission

40

- HTTP methods
 - ▣ How to submit data in form
 - ▣ GET vs. POST: side-effect

Methods in HTTP/1.0

41

- HTTP methods are operations on resources
- HTTP/1.0 supports three methods
 - ▣ GET – to retrieve a representation of a resource
 - ▣ HEAD – to retrieve only the metadata of a resource
 - ▣ POST – to submit some data to a resource (program) in web server for processing
- Note: new methods in HTTP/1.1 to be discussed in later chapter

GET method

42

- **GET** retrieves a representation of a resource
 - ▣ Usually **empty body** in **request** message
 - ▣ Browsers mainly use this method to retrieve web resources
 - ▣ Usually a read-only operation

```
GET /a.txt HTTP/1.1  
Host: example.com
```

request

```
HTTP/1.1 200 OK  
Content-Type: text/plain  
Content-Length: 5
```

Hello

response

HEAD method

43

- **HEAD** is similar to GET, but the response only contains headers and an empty body
- Useful to check the characteristics of a resource without retrieving it, e.g.
 - ▣ What is its size?
 - ▣ Is the resource still available?

```
HEAD /a.txt HTTP/1.1  
Host: example.com
```

request

```
HTTP/1.1 200 OK  
Content-Type: text/plain  
Content-Length: 5
```

response

POST method

44

- **POST** submits some data to a resource for processing
 - ▣ The resource is usually a server-side program
 - ▣ The data are usually encoded in the body of the POST request
 - ▣ Usually invokes some server-side actions

```
POST /accounts/ServiceLoginAuth?service=mail HTTP/1.1
```

```
Host: www.google.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 194
```

```
...&Email=username&Passwd=password&...
```

HTTP request to login gmail.com

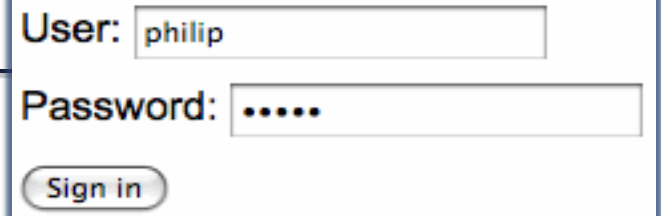
Browser usage of HTTP methods

45

- Browsers mainly use GET to retrieve web pages (HTML) and images.
- Browsers can use either GET or POST to submit HTML form.
- Web browsers assume that GET has no side-effect, but POST may have side-effect. **Important implication.**



```
<form action="http://example.com/login.jsp" method="get">  
<p>User: <input type="text" name="user" /> <br/>  
  Password: <input type="password" name="passwd" /> <br/>  
  <input type="submit" value="Sign in"/></p>  
</form>
```



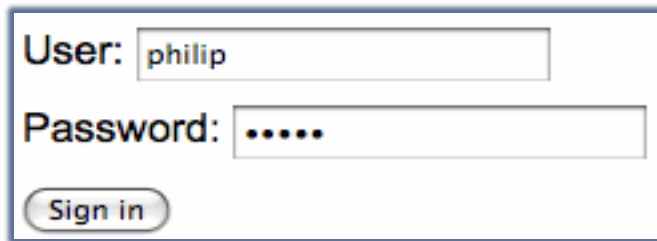
User:

Password:

Submit form by GET

46

- When a <form> is submitted by GET, form data is encoded as **x-www-form-urlencoded** and appended to the URL as query.
- Form data are serialized as name-value pairs separated by '&'
- Non-ascii characters are written as hexadecimal of UTF-8



User:

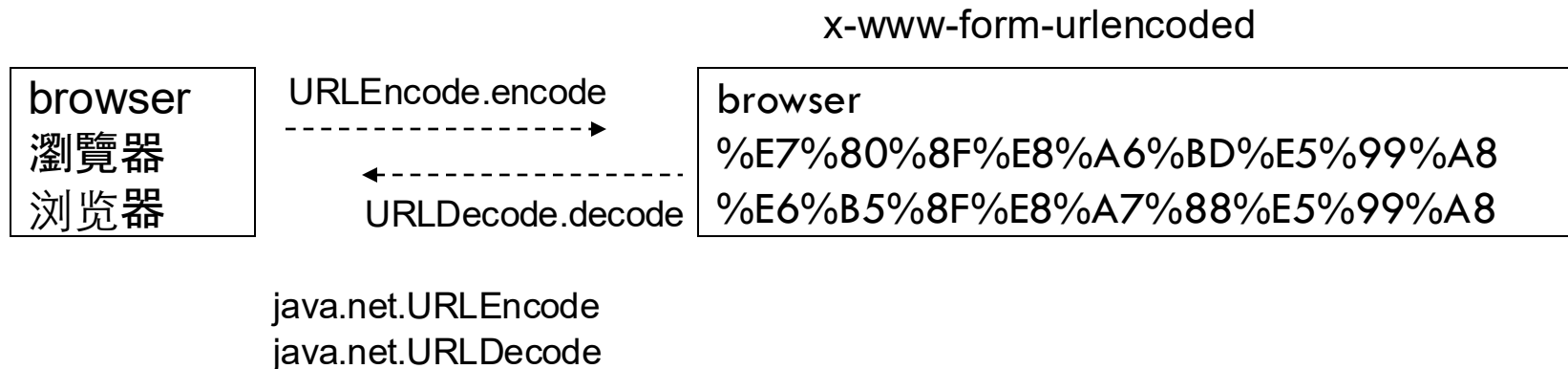
Password:

<http://example.com/login.jsp?user=philip&passwd=12345>

x-www-form-urlencoded

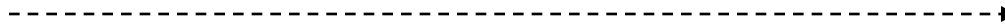
47

- x-www-form-urlencoded is used in both GET and POST requests to encode form data.
- It is necessary because URL can **only** use US-ASCII charset



Example: form submitted by GET

48



example.com



HTTP request: retrieve the resource at
`http://example.com/login.jsp?user=philip&passwd=12345`

GET /login.jsp?user=philip&passwd=12345 HTTP/1.1
Host: example.com

```
<form action="http://example.com/login.jsp" method="get">  
  ...<input name="user" /> ... <input name="passwd" /> ...  
</form>
```


Observation

49

- The GET request in form submission can be ...
 - ▣ saved in bookmark
 - ▣ saved in browser history (possible security problem!)
 - ▣ used in a hyperlink `<a href=“..”>`. When users click the link, the browser sends the same GET request as in submitting the HTML form.

```
GET /login.jsp?user=philip&passwd=12345 HTTP/1.1  
Host: example.com
```

Request for
`http://example.com/login.jsp?user=philip&passwd=12345`

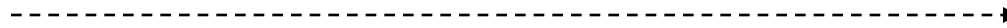
Submit form by POST

50

- When a form is submitted by POST, form data are encoded and transferred in the request body.
 - The URL *does not* include form data
 - Simple form data uses x-www-form-urlencoded
 - POST supports file upload with the MIME-type multipart/form-data
- Ref. <http://www.ietf.org/rfc/rfc1867.txt>

Example: form submitted by POST

51



example.com



HTTP request: submit this data to the program at
`http://example.com/login.jsp`

POST /login.jsp HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 24

user=philip&passwd=12345

```
<form action="http://example.com/login.jsp" method="post">
  ...<input name="user" /> ... <input name="passwd" /> ...
</form>
```

Side-effect

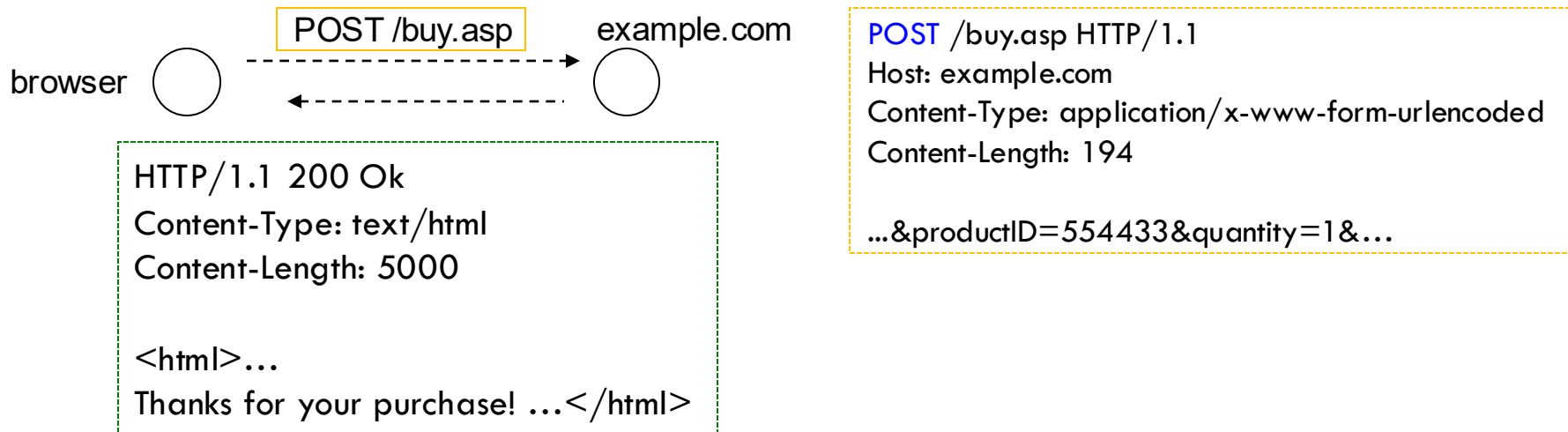
52

- GET and HEAD have no side-effect
 - ▣ read-only operations in web servers
 - ▣ A browser may repeat sending these requests without user confirmation
 - ▣ May repeat a GET request in case of network error
- POST may have side-effect
 - ▣ May cause some actions in the server (e.g. adding a record, sending an email, placing an order)
 - ▣ A browser must confirm with users before resending such requests

Example: resubmit a POST request

53

- In this example, a purchase request is sent as a POST (because it has side-effect).
- The server returns the transaction result in the response
- If the user refreshes the result page, the browser would need to send the POST request twice.
 - ▣ What would happen?



Confirmation when resubmitting request

54

- Because GET causes no action in the server, it should make no difference to repeat or skip a GET request
 - ▣ The browser may resubmit a GET request
- POST may have side-effect. If the browser resubmits a POST request, the server may perform some action twice.
 - ▣ The browser must confirm with users before resubmitting a POST request
 - ▣ *Users may be confused by the resend warning ...
Solution: Post/Redirect/Get pattern*

POSTDATA resend warning

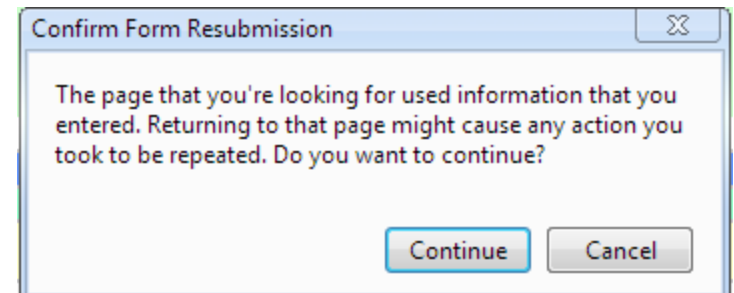
55



Resend warning in Firefox



Resend warning in Internet Explorer



Resend warning in Google Chrome

Comparison: GET and POST in form submission

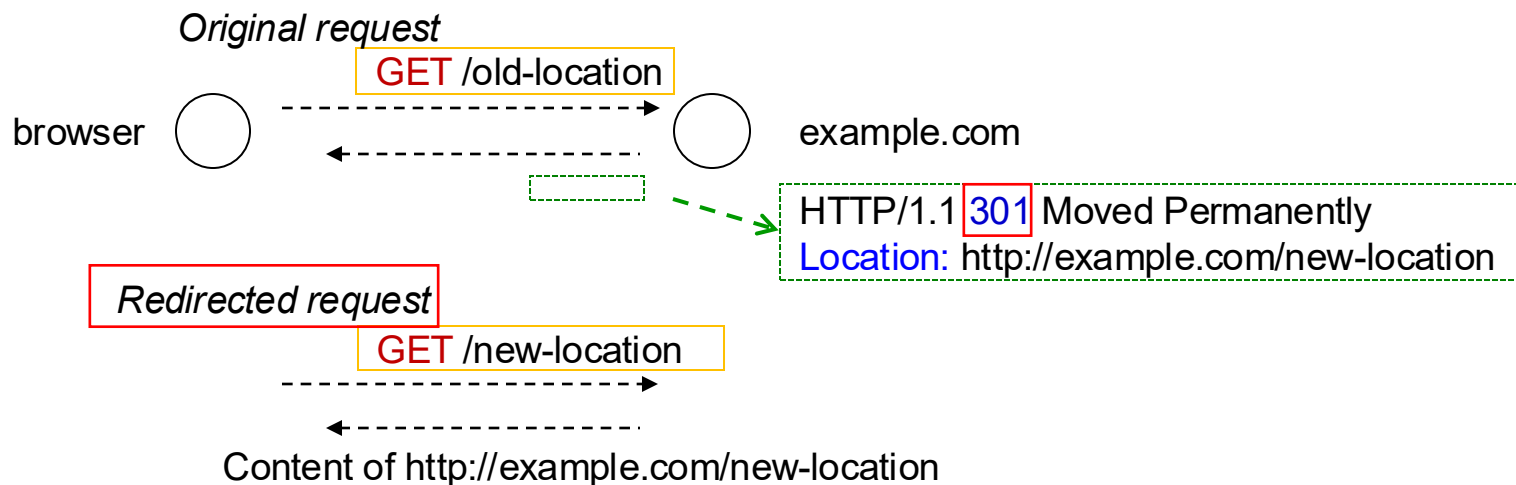
56

	GET	POST
Typical usage	Readonly query like Google search, Google Charts	Request that triggers action on server side. E.g. place an order, login, save a file
Side-effect?	Read-only operations in servers. Browsers can resubmit requests without confirmation	May cause write operations in servers. Browsers must confirm with users.
Form data in URL?	Yes. The query URL can be saved in bookmark and hyperlinks.	No
Support file upload	No	Yes

Part C. Redirection

57

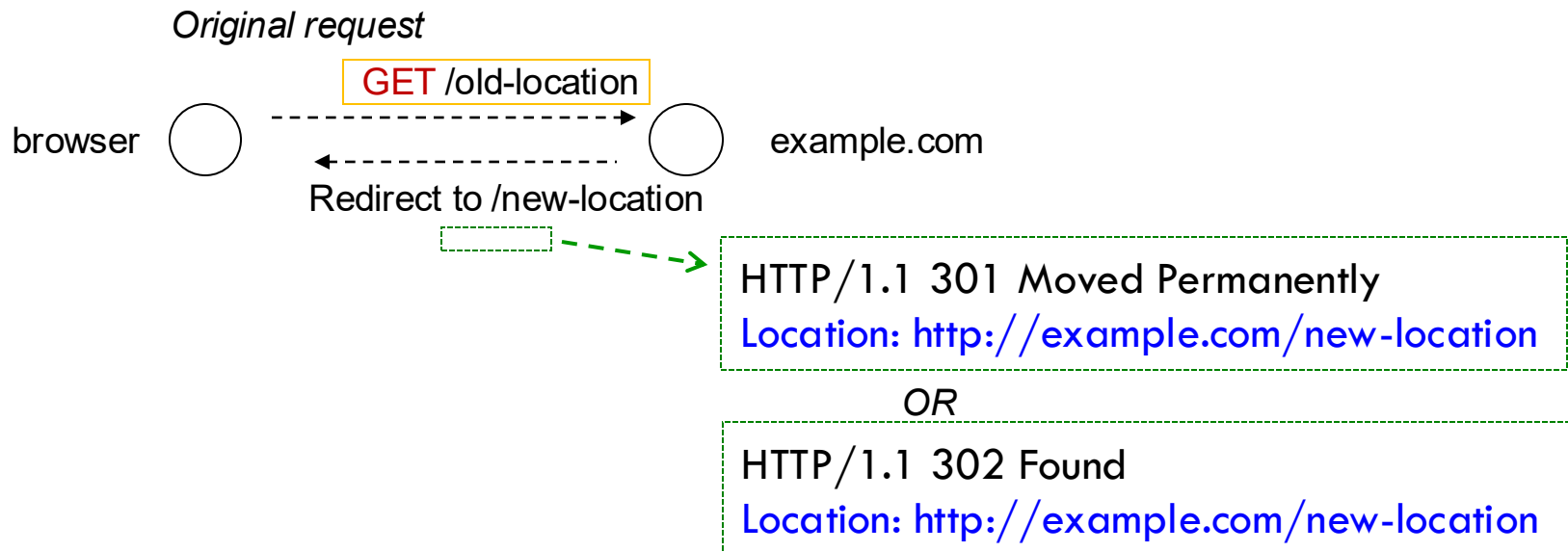
- A web server requires the browser to send a second request to get the 'real' response
- A redirection response uses status code **301**, **302**, **303** or **307** and indicates the new URL in the **Location** header in the response.
- The client has to send another request to the new URL
- *Different handling of GET and POST requests...*



Redirection of GET requests

58

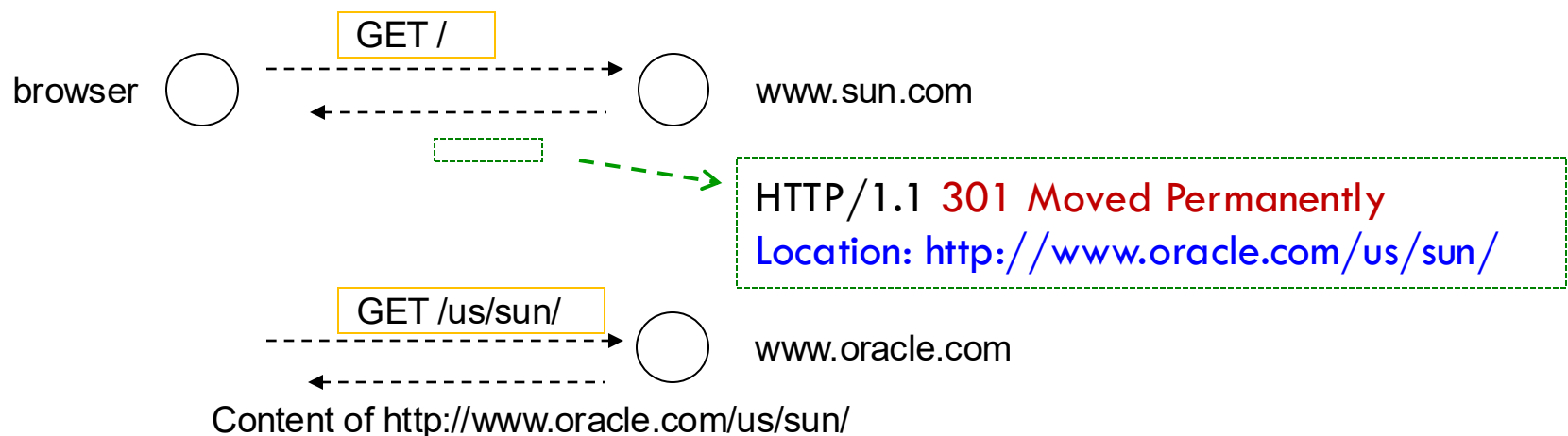
- A GET request has no side-effect. And a browser should use GET for the redirected request
- Two cases:
 - ▣ Permanent redirection
 - ▣ Temporary redirection



301 Moved permanently

59

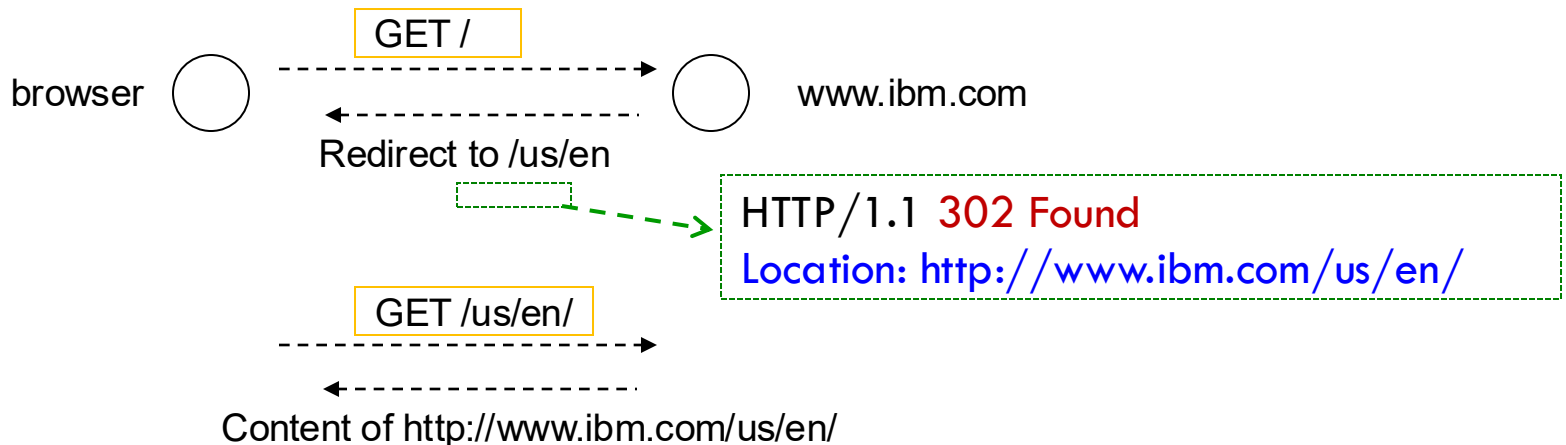
- A resource is moved permanently. A client should use the new URL for future requests.
- Redirection responses may be reused by browser cache and cache servers
- Examples: move to a different folder, to a different site



302 Found

60

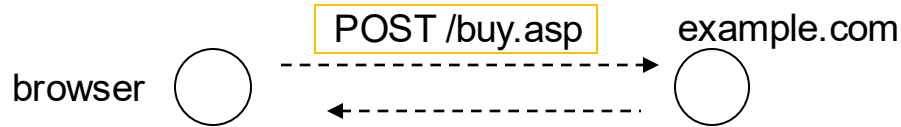
- A resource is moved temporarily. A client should use the original URL for future requests.
 - ▣ Similar status code: 303, 307
- Examples:
 - ▣ Redirection to pages of local language
 - ▣ Login before accessing mailbox
 - ▣ The resource has more than one URLs, and the server wants the user to request the canonical URL.



Redirection of a POST request

61

- Should the browser POST the data again in the redirected request?
- ➡ □ Has the server processed the form data?



HTTP/1.1 30x Some kind of redirection ...
Location: <http://example.com/page2>

POST /buy.asp HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 194
...&productID=554433&quantity=1&...

----->

POST /page2
...&productID=554433&quantity=1&...

or

GET /page2

Redirection of a POST request

62

- If the server has NOT processed the original POST request
 - ▣ the server returns status code **302 Found**
 - ▣ The client should resubmit the POST request at the new URL.

- If the server has processed the original POST request
 - ▣ the server returns status code **303 See other**
 - ▣ the client uses GET in redirected request to fetch the result of processing.

Original design ...

Many browsers got it wrong ...

63

- If the server has NOT processed the original POST request
 - ▣ the server returns status code **302 Found**
 - ▣ A browser should resubmit the POST request at the new URL, however many browser send a GET request instead.
Mistake!
 - ▣ Later revision of HTTP standard provides a new status code **307 Temporary redirect** to emphasize correct handling of resubmitting POST requests.

Original design ...

Redirection of a POST request

64

- If the server has NOT processed the original POST request
 - ▣ the server returns status code **307 Temporary redirect**
 - ▣ The client should resubmit the POST request at the new URL.

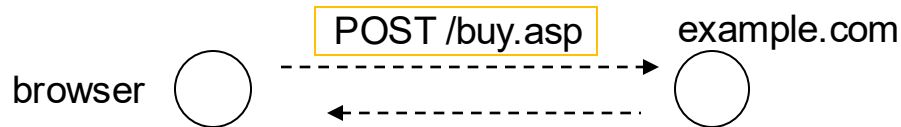
- If the server has processed the original POST request
 - ▣ the server returns status code **303 See other** or **302 Found**
 - ▣ the client uses GET in redirected request to fetch the result of processing.

Updated HTTP specification

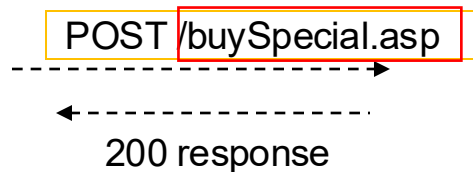
307 Temporary redirect

65

- The client has to submit the form data again at the new URL using the original HTTP method
 - ▣ The server has NOT processed the form data yet.
 - ▣ If the original request is POST, the browser POSTs the data again at the new URL.



HTTP/1.1 307 Temporary redirect
Location: <http://example.com/buySpecial.asp>



```
POST /buy.asp HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 194

...&productID=554433&quantity=1&...
```

```
POST /buySpecial.asp HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 194

...&productID=554433&quantity=1&...
```

303 See other

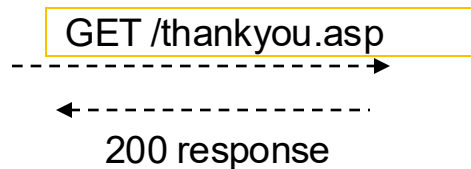
66

- A client should submit a **GET** request for the new URL in a 303 response, regardless of the method in the original request
 - ▣ The server has **already processed the original request**
 - ▣ The form data in the original request, (if any), is **not sent** in the redirected request.



```
POST /buy.asp HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 194

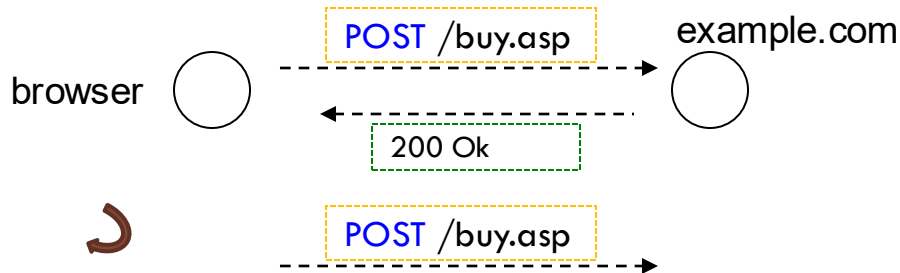
...&productID=554433&quantity=1&...
```



```
GET /thankyou.asp HTTP/1.1
Host: example.com
```

Reloading a response from POST

67



POST /buy.asp HTTP/1.1

...

...&productID=554433&quantity=1&...

HTTP/1.1 200 Ok

Content-Type: text/html

Content-Length: 5000

<html>...

Thanks for your purchase! ...</html>

- When a user reloads a page that is a response to a previous POST request,
 - The browser would need to resend the POST request
 - The browser needs to confirm whether the user really wants to do the transaction one more time
 - Usually NO
 - It is confusing to users

Confirm Form Resubmission

The page that you're looking for used information that you entered. Returning to that page might cause any action you took to be repeated. Do you want to continue?

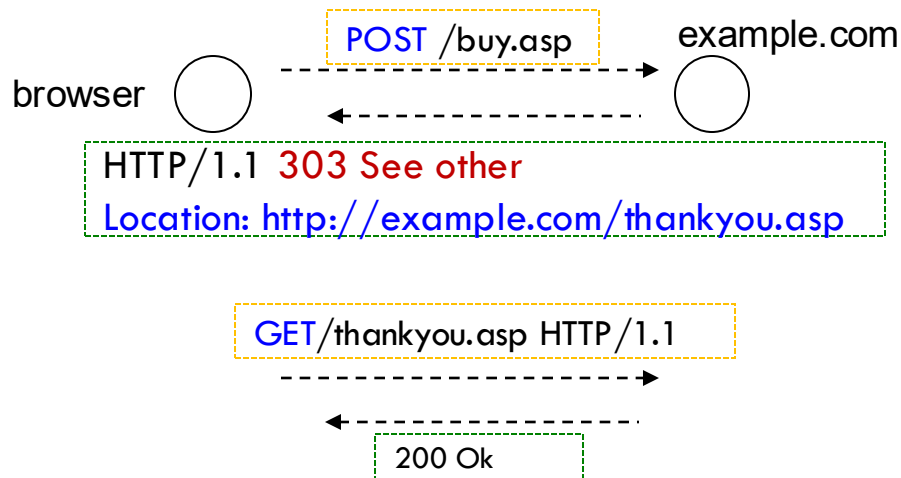
Continue

Cancel

Post/redirect/get pattern

68

- The server does not directly return the transaction result page of the POST request.
- Instead, the browser is redirected to the result page.
- The browser loads the result page with GET. When the user refreshes the page, there is no confusion at the resend warning page.



```
POST /buy.asp HTTP/1.1
...
...&productID=554433&quantity=1&...
```

```
HTTP/1.1 200 Ok
Content-Type: text/html
Content-Length: 5000

<html>...
Thanks for your purchase! ...</html>
```

Common code for redirection

69

Status code	Meaning
301 Moved permanently	The client should use the new URL for future requests
302 Found	Supposed to work like 307, but browsers implement it like 303.
303 See other	The request has been processed, but the server wants the client to send another GET request to the new URL for a response document.
304 Not Modified	Used in conditional GET
307 Temporary Redirect	The request has NOT been processed. The client needs to resubmit the request to the server at the new URL. (probably POST again)

Exercise

70

- Check the redirection responses in the following
 - ▣ <http://www.gmail.com/>
 - ▣ <http://www.hotmail.com/>
 - ▣ <http://www.macromedia.com/>
 - ▣ <http://www.mpu.edu.mo/>
- You can use Developer tools, or <http://web-sniffer.net/> to examine the responses.
- What are the purposes of redirection?